



# 拥塞控制

## ✍ 问题

部分或整个子网有太多的数据而造成性能下降的状况。

## ✍ 原因

内存配置不当、处理器太慢、带宽太低、缺乏综合治理等，本质原因是： $\sum \text{对资源的需求} > \text{可用资源}$ 。不能头痛医头，脚痛医脚，只有等网络中所有组件都相互平衡后，才能消除拥塞。



# 拥塞控制思想

✎ **思想：**控制发送方发送速率

发送端的窗口大小不仅取决于接收端（接收端通知的窗口大小），而且取决于网络的拥塞状况（拥塞窗口的大小），即：

**发送窗口 = Min[通知窗口, 拥塞窗口]**

✎ **通知窗口 (rwnd)：**是接收端根据其接收能力许诺的窗口值，是来自接收端的流量控制。接收端将通知窗口的值放在TCP报文段的首部中，传送给发送端。

✎ **拥塞窗口 (cwnd)：**是发送端根据网络拥塞情况得出的窗口值，是来自发送端的拥塞控制。



# 拥塞窗口的确定

## (AIMD:加法增大乘法减小)

 慢开始和拥塞避免

 快重传和快恢复



# 慢开始和拥塞避免

## ✎ 增加策略

当一个连接初始化时，拥塞窗口设为1（一个报文段）。在拥塞发生前，拥塞窗口从1开始，在每收到一个对新的报文段的确认后，将拥塞窗口增加1（慢开始），即拥塞窗口以指数规律进行增长（1、2、4、8...）。当拥塞窗口大小到达门限（**ssthresh**）时，以线性规律进行增长（避免拥塞，加法增大）。

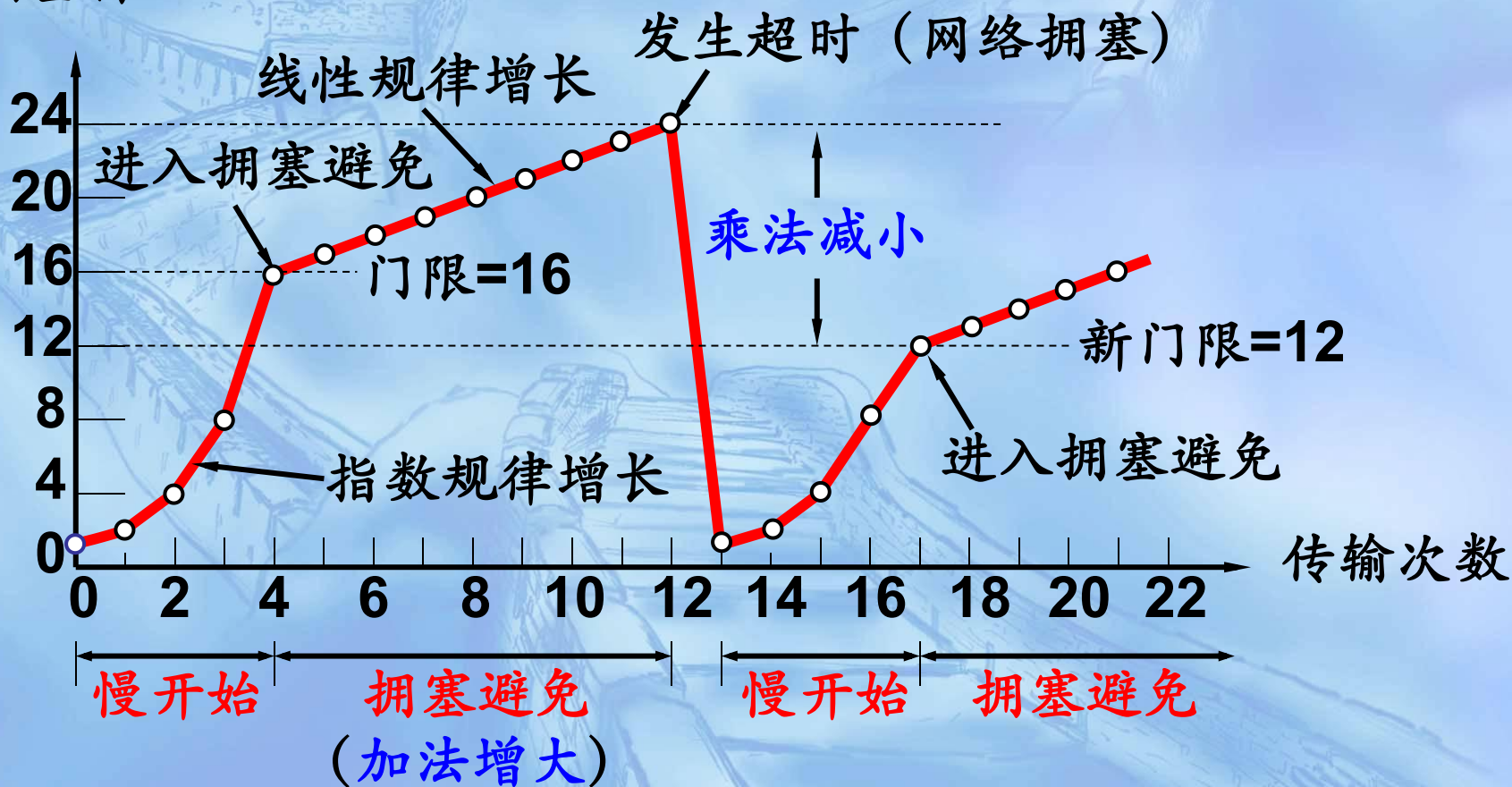
## ✎ 减少策略

如果出现拥塞（判断的依据是超时），窗口门限值设置为上次拥塞窗口大小的一半（乘法减小），而拥塞窗口降低到1，又从1开始增加。



# 图示

## 拥塞窗口cwnd



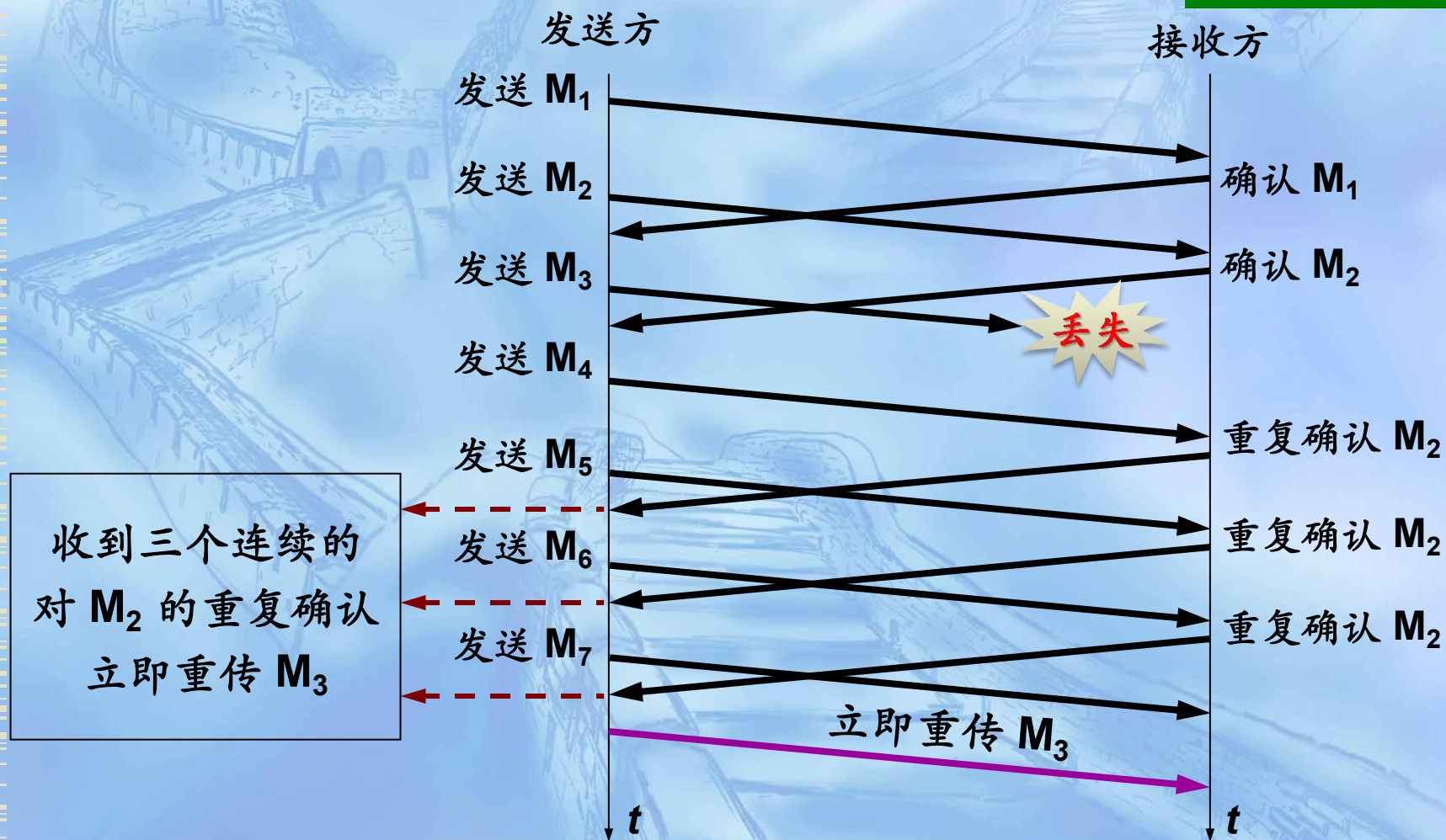


# 快重传

- ✎ 快重传算法首先要求接收方每收到一个失序的报文段后就立即发出重复确认。这样做可以让发送方及早知道有报文段没有到达接收方；
- ✎ 发送方只要一连续收到三个重复确认就应当立即重传对方尚未收到的报文段；
- ✎ 不难看出，快重传并非取消重传计时器，而是在某些情况下可更早地重传丢失的报文段，提高网络的吞吐量。



## 例:快重传



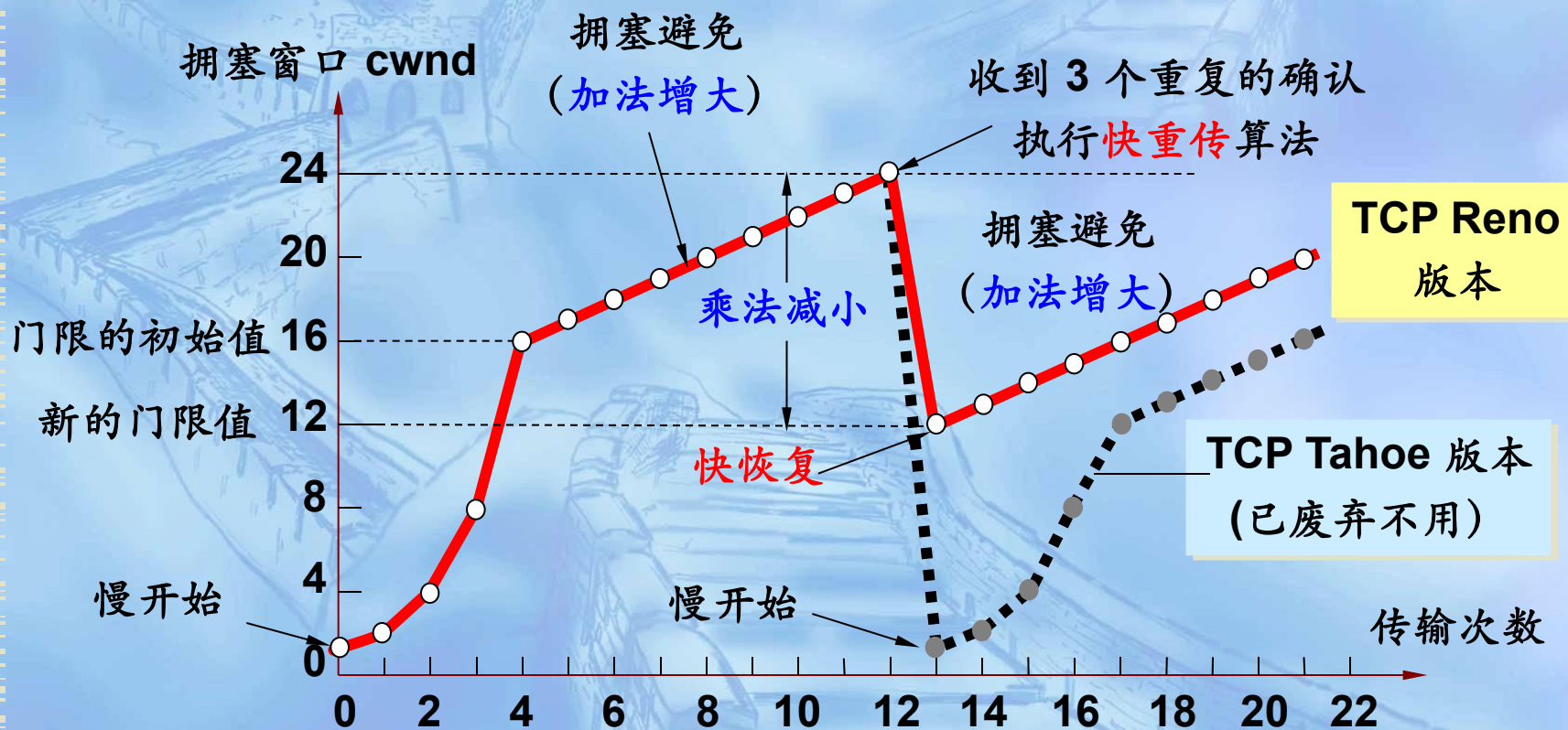


# 快恢复

- ✎ 当发送端收到连续三个重复的确认时，就执行“乘法减小”算法，把慢开始门限（**ssthresh**）减半。但接下去不执行慢开始算法；
- ✎ 由于发送方现在认为网络很可能没有发生拥塞，因此现在不执行慢开始算法，即拥塞窗口 **cwnd** 现在不设置为 1，而是设置为慢开始门限减半后的数值，然后开始执行拥塞避免算法（加法增大），使拥塞窗口缓慢地线性增大（快恢复）。



# 图示





# TCP拥塞控制流程图

